

Week 4 Day 4 - Docker Compose (Code & Implementation)

What You'll Build Today

Replace multiple `docker run` commands with a single `docker-compose.yml` file.

Section 1: Basic Docker Compose

What Docker Compose Replaces

Before (Day 3 - Multiple commands):

```
bash

docker network create myapp-network
docker volume create postgres-data
docker volume create redis-data

docker run -d --name postgres --network myapp-network \
-e POSTGRES_PASSWORD=secret -v postgres-data:/var/lib/postgresql/data \
postgres:15-alpine

docker run -d --name redis --network myapp-network \
-v redis-data:/data redis:7-alpine

docker run -d --name api --network myapp-network \
-p 8080:8080 -e DB_HOST=postgres -e REDIS_HOST=redis myapp:v1
```

After (Day 4 - One file, one command):

`docker-compose.yml`:

```
yaml
```

version: '3.8'

services:

postgres:

image: postgres:15-alpine

environment:

POSTGRES_PASSWORD: secret

POSTGRES_DB: myapp

volumes:

- postgres-data:/var/lib/postgresql/data

networks:

- myapp-network

redis:

image: redis:7-alpine

volumes:

- redis-data:/data

networks:

- myapp-network

api:

build: ./api

ports:

- "8080:8080"

environment:

DB_HOST: postgres

REDIS_HOST: redis

depends_on:

- postgres

- redis

networks:

- myapp-network

volumes:

postgres-data:

redis-data:

networks:

myapp-network:

Run everything:

bash

`docker-compose up -d`

Section 2: Complete API + Database Setup

Project Structure

```
project/
├── docker-compose.yml
├── .env
├── api/
│   ├── main.go
│   ├── go.mod
│   ├── go.sum
│   └── Dockerfile
└── README.md
```

API Code with Health Checks

api/main.go:

```
go
```

```
package main
```

```
import (
```

```
    "context"
```

```
    "database/sql"
```

```
    "encoding/json"
```

```
    "fmt"
```

```
    "log"
```

```
    "net/http"
```

```
    "os"
```

```
    "time"
```

```
    _ "github.com/lib/pq"
```

```
    "github.com/redis/go-redis/v9"
```

```
)
```

```
var (
```

```
    db      *sql.DB
```

```
    redisClient *redis.Client
```

```
    ctx      = context.Background()
```

```
)
```

```
type HealthStatus struct {
```

```
    Status string `json:"status"`
```

```
    Database string `json:"database"`
```

```
    Redis string `json:"redis"`
```

```
    Version string `json:"version"`
```

```
}
```

```
type User struct {
```

```
    ID int `json:"id"`
```

```
    Name string `json:"name"`
```

```
    Email string `json:"email"`
```

```
}
```

```
func initDB() error {
```

```
    connStr := fmt.Sprintf(
```

```
        "host=%s port=%s user=%s password=%s dbname=%s sslmode=disable",
```

```
        getEnv("DB_HOST", "localhost"),
```

```
        getEnv("DB_PORT", "5432"),
```

```
        getEnv("DB_USER", "postgres"),
```

```
        getEnv("DB_PASSWORD", "secret"),
```

```
        getEnv("DB_NAME", "myapp"),
```

```
)
```

```
var err error
```

```
db, err = sql.Open("postgres", connStr)
if err != nil {
    return err
}

// Retry connection with timeout
for i := 0; i < 30; i++ {
    err = db.Ping()
    if err == nil {
        break
    }
    log.Printf("Waiting for database... (%d/30)", i+1)
    time.Sleep(1 * time.Second)
}
if err != nil {
    return err
}

log.Println("Database connected")

// Create table
createTable := `
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
)`
_, err = db.Exec(createTable)
return err
}

func initRedis() error {
    redisClient = redis.NewClient(&redis.Options{
        Addr: fmt.Sprintf("%s:%s",
            getEnv("REDIS_HOST", "localhost"),
            getEnv("REDIS_PORT", "6379"),
        ),
        Password: getEnv("REDIS_PASSWORD", ""),
        DB:      0,
    })
}

// Test connection
_, err := redisClient.Ping(ctx).Result()
if err != nil {
    log.Printf("Redis not available: %v", err)
    return err
}
```

```

    }

    log.Println("Redis connected")
    return nil
}

func getEnv(key, defaultVal string) string {
    if val := os.Getenv(key); val != "" {
        return val
    }
    return defaultVal
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    health := HealthStatus{
        Status: "healthy",
        Version: "1.0.0",
    }

    // Check database
    if err := db.Ping(); err != nil {
        health.Database = "unhealthy"
        health.Status = "degraded"
    } else {
        health.Database = "healthy"
    }

    // Check Redis
    if _, err := redisClient.Ping(ctx).Result(); err != nil {
        health.Redis = "unhealthy"
        health.Status = "degraded"
    } else {
        health.Redis = "healthy"
    }

    w.Header().Set("Content-Type", "application/json")
    if health.Status != "healthy" {
        w.WriteHeader(http.StatusServiceUnavailable)
    }
    json.NewEncoder(w).Encode(health)
}

func getUsersHandler(w http.ResponseWriter, r *http.Request) {
    // Try cache
    cached, err := redisClient.Get(ctx, "users:all").Result()
    if err == nil {
        w.Header().Set("Content-Type", "application/json")
    }
}

```

```

w.Header().Set("X-Cache", "HIT")
w.Write([]byte(cached))
return
}

// Query database
rows, err := db.Query("SELECT id, name, email FROM users ORDER BY id")
if err != nil {
    http.Error(w, err.Error(), http.StatusInternalServerError)
    return
}
defer rows.Close()

users := []User{}
for rows.Next() {
    var user User
    if err := rows.Scan(&user.ID, &user.Name, &user.Email); err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
    users = append(users, user)
}

// Cache for 30 seconds
data, _ := json.Marshal(users)
redisClient.Set(ctx, "users:all", data, 30*time.Second)

w.Header().Set("Content-Type", "application/json")
w.Header().Set("X-Cache", "MISS")
w.Write(data)
}

func createUserHandler(w http.ResponseWriter, r *http.Request) {
    var user User
    if err := json.NewDecoder(r.Body).Decode(&user); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    err := db.QueryRow(
        "INSERT INTO users (name, email) VALUES ($1, $2) RETURNING id",
        user.Name, user.Email,
    ).Scan(&user.ID)

    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

```

```

}

// Invalidate cache
redisClient.Del(ctx, "users:all")

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)
json.NewEncoder(w).Encode(user)
}

func main() {
    if err := initDB(); err != nil {
        log.Fatal("Database initialization failed:", err)
    }
    defer db.Close()

    if err := initRedis(); err != nil {
        log.Println("Redis initialization failed:", err)
    }

    http.HandleFunc("/health", healthHandler)
    http.HandleFunc("/users", func(w http.ResponseWriter, r *http.Request) {
        switch r.Method {
        case http.MethodGet:
            getUsersHandler(w, r)
        case http.MethodPost:
            createUserHandler(w, r)
        default:
            http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        }
    })

    port := getEnv("PORT", "8080")
    log.Printf("Server starting on port %s", port)
    log.Fatal(http.ListenAndServe(":"+port, nil))
}

```

api/go.mod:

```
go
```



```
module myapp
```

```
go 1.21
```

```
require (  
    github.com/lib/pq v1.10.9  
    github.com/redis/go-redis/v9 v9.3.0  
)
```

api/Dockerfile:

```
dockerfile
```

```
FROM golang:1.21-alpine AS builder
```

```
WORKDIR /app
```

```
COPY go.mod go.sum ./
```

```
RUN go mod download
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 GOOS=linux go build -o main .
```

```
FROM alpine:latest
```

```
RUN apk --no-cache add ca-certificates curl
```

```
WORKDIR /root/
```

```
COPY --from=builder /app/main .
```

```
EXPOSE 8080
```

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \  
    CMD curl -f http://localhost:8080/health || exit 1
```

```
CMD ["/main"]
```

Section 3: Docker Compose with Health Checks

docker-compose.yml:

```
yaml
```

version: '3.8'

services:

PostgreSQL Database

postgres:

image: postgres:15-alpine

container_name: myapp-postgres

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: \${DB_PASSWORD:-secret}

POSTGRES_DB: myapp

volumes:

- postgres-data:/var/lib/postgresql/data

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- myapp-network

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 10s

timeout: 5s

retries: 5

restart: unless-stopped

Redis Cache

redis:

image: redis:7-alpine

container_name: myapp-redis

command: redis-server --appendonly yes

volumes:

- redis-data:/data

networks:

- myapp-network

healthcheck:

test: ["CMD", "redis-cli", "ping"]

interval: 10s

timeout: 3s

retries: 5

restart: unless-stopped

API Service

api:

build:

context: ./api

dockerfile: Dockerfile

container_name: myapp-api

ports:

```
- "${API_PORT:-8080}:8080"
```

environment:

```
DB_HOST: postgres
```

```
DB_PORT: 5432
```

```
DB_USER: postgres
```

```
DB_PASSWORD: ${DB_PASSWORD:-secret}
```

```
DB_NAME: myapp
```

```
REDIS_HOST: redis
```

```
REDIS_PORT: 6379
```

```
PORT: 8080
```

depends_on:

postgres:

```
condition: service_healthy
```

redis:

```
condition: service_healthy
```

networks:

```
- myapp-network
```

```
restart: unless-stopped
```

volumes:

postgres-data:

```
driver: local
```

redis-data:

```
driver: local
```

networks:

myapp-network:

```
driver: bridge
```

Environment File (.env):

```
bash
```

```
# Database
```

```
DB_PASSWORD=mysecretpassword
```

```
# API
```

```
API_PORT=8080
```

```
# Optional: Override compose project name
```

```
COMPOSE_PROJECT_NAME=myapp
```

Optional Init SQL (init.sql):

```
sql
```

-- Create initial data

```
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Insert sample data

```
INSERT INTO users (name, email) VALUES  
  ('Alice', 'alice@example.com'),  
  ('Bob', 'bob@example.com')  
ON CONFLICT (email) DO NOTHING;
```

Section 4: Advanced Compose Features

Development vs Production Configs

docker-compose.yml (Base):

yaml

version: '3.8'

services:

postgres:

image: postgres:15-alpine

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: secret

POSTGRES_DB: myapp

volumes:

- postgres-data:/var/lib/postgresql/data

networks:

- myapp-network

redis:

image: redis:7-alpine

networks:

- myapp-network

api:

build: ./api

ports:

- "8080:8080"

environment:

DB_HOST: postgres

REDIS_HOST: redis

depends_on:

- postgres

- redis

networks:

- myapp-network

volumes:

postgres-data:

networks:

myapp-network:

docker-compose.dev.yml (Development overrides):

yaml

version: '3.8'

services:

api:

build:

context: ./api

dockerfile: Dockerfile

target: builder *# Stop at builder stage for debugging*

volumes:

- ./api:/app *# Mount source code for hot reload*

environment:

LOG_LEVEL: debug

DEBUG: "true"

command: go run main.go *# Use go run for development*

postgres:

ports:

- "5432:5432" *# Expose for external tools*

redis:

ports:

- "6379:6379" *# Expose for redis-cli*

docker-compose.prod.yml (Production overrides):

yaml

version: '3.8'

services:

api:

image: myapp-api:\${VERSION:-latest}

deploy:

replicas: 3

resources:

limits:

cpus: '0.5'

memory: 512M

reservations:

cpus: '0.25'

memory: 256M

restart: always

logging:

driver: "json-file"

options:

max-size: "10m"

max-file: "3"

postgres:

environment:

POSTGRES_PASSWORD_FILE: /run/secrets/db_password

secrets:

- db_password

restart: always

redis:

restart: always

secrets:

db_password:

file: ./secrets/db_password.txt

Usage:

bash

Development

docker-compose -f docker-compose.yml -f docker-compose.dev.yml up

Production

docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d

Section 5: Complete Makefile for Compose

Makefile:

```
makefile
```


.PHONY: help build up down restart logs ps test clean dev prod

Default target

.DEFAULT_GOAL := help

Load environment variables

include .env

export

Help target

help:

@echo "Available targets:"

@echo " make build - Build all services"

@echo " make up - Start all services"

@echo " make down - Stop all services"

@echo " make restart - Restart all services"

@echo " make logs - View logs"

@echo " make ps - List running services"

@echo " make test - Run API tests"

@echo " make clean - Remove all containers and volumes"

@echo " make dev - Start in development mode"

@echo " make prod - Start in production mode"

Build all services

build:

docker-compose build

Start all services

up:

docker-compose up -d

@echo "Waiting for services to be healthy..."

@sleep 5

@docker-compose ps

Stop all services

down:

docker-compose down

Restart all services

restart: down up

View logs

logs:

docker-compose logs -f

Show service status

ps:

docker-compose ps

Run tests

test:

@echo "Testing health endpoint..."

@curl -f http://localhost:8080/health || exit 1

@echo "\nCreating test user..."

@curl -X POST http://localhost:8080/users \

-H "Content-Type: application/json" \

-d '{"name":"Test","email":"test@example.com"}'

@echo "\nListing users..."

@curl http://localhost:8080/users

@echo "\n✅ All tests passed!"

Clean everything

clean:

docker-compose down -v

docker system prune -f

Development mode

dev:

docker-compose -f docker-compose.yml -f docker-compose.dev.yml up

Production mode

prod:

docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d

Database shell

db-shell:

docker-compose exec postgres psql -U postgres -d myapp

Redis CLI

redis-cli:

docker-compose exec redis redis-cli

API shell

api-shell:

docker-compose exec api sh

View API logs only

api-logs:

docker-compose logs -f api

Rebuild and restart specific service

rebuild-api:

```
docker-compose up -d --build api
```

Section 6: Compose Commands Reference

Essential Commands

bash

Start services

<code>docker-compose up</code>	<i># Foreground</i>
<code>docker-compose up -d</code>	<i># Background (detached)</i>
<code>docker-compose up --build</code>	<i># Rebuild before starting</i>

Stop services

<code>docker-compose stop</code>	<i># Stop containers</i>
<code>docker-compose down</code>	<i># Stop and remove containers</i>
<code>docker-compose down -v</code>	<i># Also remove volumes</i>

Build

<code>docker-compose build</code>	<i># Build all services</i>
<code>docker-compose build api</code>	<i># Build specific service</i>
<code>docker-compose build --no-cache</code>	<i># Build without cache</i>

View status

<code>docker-compose ps</code>	<i># List containers</i>
<code>docker-compose ps -a</code>	<i># Include stopped containers</i>
<code>docker-compose top</code>	<i># Show running processes</i>

Logs

<code>docker-compose logs</code>	<i># All logs</i>
<code>docker-compose logs -f</code>	<i># Follow logs</i>
<code>docker-compose logs api</code>	<i># Specific service</i>
<code>docker-compose logs --tail=50</code>	<i># Last 50 lines</i>

Execute commands

<code>docker-compose exec api sh</code>	<i># Shell in API container</i>
<code>docker-compose exec postgres psql</code>	<i># Postgres CLI</i>
<code>docker-compose run api go test</code>	<i># Run one-off command</i>

Scale services

<code>docker-compose up -d --scale api=3</code>	<i># Run 3 API instances</i>
---	------------------------------

Restart

<code>docker-compose restart</code>	<i># Restart all</i>
<code>docker-compose restart api</code>	<i># Restart specific service</i>

Section 7: Testing Script

test-compose.sh:

```
bash
```

```
#!/bin/bash
```

```
set -e
```

```
echo "=====
```

```
echo "Testing Docker Compose Setup"
```

```
echo "=====
```

```
# Colors
```

```
GREEN="\033[0;32m'
```

```
RED="\033[0;31m'
```

```
NC="\033[0m' # No Color
```

```
# Test 1: Check services are running
```

```
echo -e "\n${GREEN} Test 1: Checking services...${NC}"
```

```
docker-compose ps
```

```
# Test 2: Health check
```

```
echo -e "\n${GREEN} Test 2: API health check...${NC}"
```

```
response=$(curl -s http://localhost:8080/health)
```

```
if echo "$response" | grep -q "healthy"; then
```

```
    echo "✅ Health check passed"
```

```
else
```

```
    echo "❌ Health check failed"
```

```
    exit 1
```

```
fi
```

```
# Test 3: Database connectivity
```

```
echo -e "\n${GREEN} Test 3: Database connectivity...${NC}"
```

```
docker-compose exec -T postgres pg_isready -U postgres
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ Database is ready"
```

```
else
```

```
    echo "❌ Database not ready"
```

```
    exit 1
```

```
fi
```

```
# Test 4: Redis connectivity
```

```
echo -e "\n${GREEN} Test 4: Redis connectivity...${NC}"
```

```
docker-compose exec -T redis redis-cli ping
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ Redis is ready"
```

```
else
```

```
    echo "❌ Redis not ready"
```

```
    exit 1
```

```
fi
```

```

# Test 5: Create user
echo -e "\n${GREEN}Test 5: Creating user...${NC}"
response=$(curl -s -X POST http://localhost:8080/users \
-H "Content-Type: application/json" \
-d '{"name":"Test User","email":"test@example.com"}')
if echo "$response" | grep -q "id"; then
    echo "✅ User created successfully"
else
    echo "❌ User creation failed"
    exit 1
fi

# Test 6: List users
echo -e "\n${GREEN}Test 6: Listing users...${NC}"
response=$(curl -s http://localhost:8080/users)
if echo "$response" | grep -q "Test User"; then
    echo "✅ User retrieved successfully"
else
    echo "❌ User retrieval failed"
    exit 1
fi

# Test 7: Cache test
echo -e "\n${GREEN}Test 7: Testing cache...${NC}"
# First request - should be cache miss
headers1=$(curl -s -I http://localhost:8080/users | grep "X-Cache")
echo "First request: $headers1"

# Second request - should be cache hit
headers2=$(curl -s -I http://localhost:8080/users | grep "X-Cache")
echo "Second request: $headers2"

if echo "$headers2" | grep -q "HIT"; then
    echo "✅ Cache working correctly"
else
    echo "⚠️ Cache not working (not critical)"
fi

echo -e "\n${GREEN}===== "
echo "All tests passed! ✅ "
echo "===== ${NC}"

```

Make executable and run:

```
bash
```

```
chmod +x test-compose.sh
./test-compose.sh
```

Section 8: GitHub Actions CI/CD

.github/workflows/docker-compose-test.yml:

yml

name: Docker Compose Test

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

steps:

- **name:** Checkout code

uses: actions/checkout@v3

- **name:** Create .env file

run: |

echo "DB_PASSWORD=testpassword" >> .env

echo "API_PORT=8080" >> .env

- **name:** Start services

run: |

docker-compose up -d

sleep 10

- **name:** Check service health

run: |

docker-compose ps

curl -f http://localhost:8080/health

- **name:** Run tests

run: ./test-compose.sh

- **name:** View logs on failure

if: failure()

run: docker-compose logs

- **name:** Stop services

run: docker-compose down -v

Section 9: VS Code DevContainer

.devcontainer/devcontainer.json:

json

```
{
  "name": "MyApp DevContainer",
  "dockerComposeFile": "../docker-compose.yml",
  "service": "api",
  "workspaceFolder": "/app",
  "customizations": {
    "vscode": {
      "extensions": [
        "golang.go",
        "ms-azuretools.vscode-docker"
      ],
      "settings": {
        "go.useLanguageServer": true
      }
    }
  },
  "forwardPorts": [8080, 5432, 6379],
  "postCreateCommand": "go mod download"
}
```

Section 10: Monitoring Stack

docker-compose.monitoring.yml:

yaml

version: '3.8'

services:

Prometheus for metrics

prometheus:

image: prom/prometheus:latest

container_name: prometheus

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus-data:/prometheus

ports:

- "9090:9090"

networks:

- myapp-network

command:

- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'

Grafana for visualization

grafana:

image: grafana/grafana:latest

container_name: grafana

ports:

- "3000:3000"

environment:

- GF_SECURITY_ADMIN_PASSWORD=admin

volumes:

- grafana-data:/var/lib/grafana

networks:

- myapp-network

depends_on:

- prometheus

volumes:

prometheus-data:

grafana-data:

networks:

myapp-network:

external: true

prometheus.yml:

yaml

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'api'
    static_configs:
      - targets: ['api:8080']
```

Start monitoring:

```
bash

docker-compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d
```

✓ End of Day 4 Code Checklist

After implementing all sections:

- ☐ **docker-compose.yml created** - Single file defining all services
- ☐ **Multi-service orchestration** - API + PostgreSQL + Redis working
- ☐ **Service dependencies** - `depends_on` with health checks
- ☐ **Health checks** - All services have health checks
- ☐ **Compose commands** - Familiar with up, down, logs, ps, etc.

Quick Start Guide

```
bash
```

1. Create project structure

```
mkdir myapp && cd myapp
```

```
mkdir api
```

2. Copy all code files (api/main.go, docker-compose.yml, etc.)

3. Create .env file

```
echo "DB_PASSWORD=mysecretpassword" > .env
```

4. Build and start

```
docker-compose up -d
```

5. Test

```
curl http://localhost:8080/health
```

6. View logs

```
docker-compose logs -f
```

7. Stop

```
docker-compose down
```

That's all the code for Day 4! Ready for the walkthrough guide when you are. 🚀